

TOTAL MODULES	CURRICULUM PART	CORE MECHANICS	REFERENCE STANDARD
64 Modules + Practice Lab	Part 24 — Memory & GC Architecture	Stack vs Heap, Mark & Sweep, WeakRef	ECMAScript 2026 / V8 GC Engine Spec

 JavaScript Memory Management & Garbage Collection — মেমরি আর্কিটেকচার শেখার মূল উদ্দেশ্য

JavaScript-এ মেমরি কীভাবে বরাদ্দ হয়, অবজেক্ট রেফারেন্স কীভাবে কাজ করে এবং মেমরি লিক এড়াতে কী কী সতর্কতা প্রয়োজন—তা জানা একজন প্রোডাকশন ইঞ্জিনিয়ারের জন্য অপরিহার্য। এই চ্যাপ্টারে আমরা **Stack vs Heap**, **Mark-and-Sweep Garbage Collection**, **Reachability Tree**, **Closure Memory**, **Detached DOM Nodes**, **WeakMap/WeakSet** এবং **Chrome DevTools Heap Profiling** গভীরভাবে শিখব।

-  1. Memory Allocation
-  2. Stack vs Heap
-  3. References & Values
-  4. Reachability & Roots
-  5. Mark-and-Sweep GC
-  6. Circular References
-  7. Memory Leak Traps
-  8. Detached DOM Nodes
-  9. WeakMap & WeakRef
-  10. Heap Profiling

Part 01 — Core Concepts, Runtime Architecture & Memory Allocation (30.1 – 30.9)

30.1 Memory Management কী?

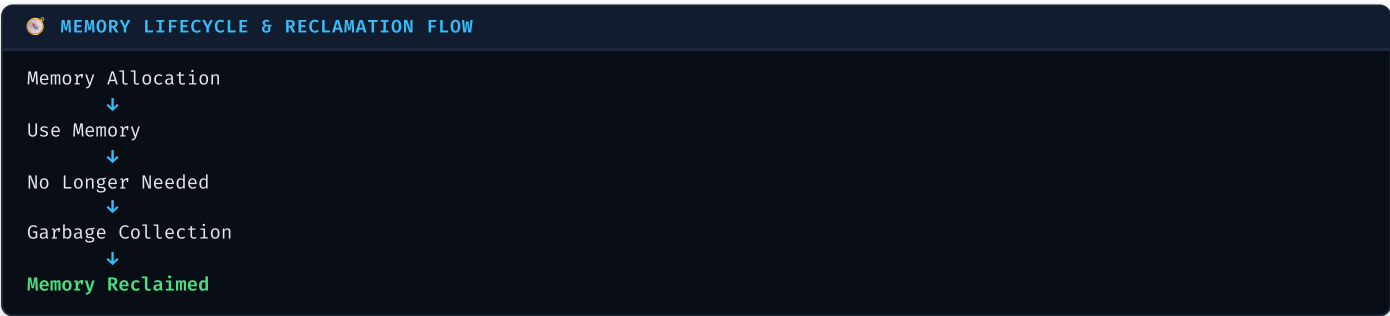
Program চলার সময় বিভিন্ন data রাখার জন্য computer memory ব্যবহার করে। যেমন:

JAVASCRIPT SOURCEJAVASCRIPT

```
const name = "Ripon";
const age = 23;

const user = {
  name: "Ripon",
  age: 23
};
```

এই data program চলার সময় memory-তে রাখা হয়। তারপর যখন data আর প্রয়োজন হয় না, JavaScript সেই memory reclaim করার ব্যবস্থা করে। এই পুরো process:



30.2 JavaScript কি নিজে Memory Manage করে?

হ্যাঁ। C/C++-এর মতো language-এ developer-কে অনেক ক্ষেত্রে manually memory manage করতে হয়। JavaScript-এ সাধারণত:

🕒 ARCHITECTURE & MEMORY GRAPH DIAGRAM

```
Developer
  ↓
Create Object
  ↓
JavaScript Runtime
  ↓
Memory Management
  ↓
Garbage Collector
```

তাই JavaScript-কে **garbage-collected language** বলা হয়।

30.3 Memory Lifecycle

একটি সাধারণ memory lifecycle:

🕒 MEMORY LIFECYCLE & RECLAMATION FLOW

```
1. Allocate
  ↓
2. Use
  ↓
3. No longer reachable
  ↓
4. Garbage Collector detects
  ↓
5. Memory reclaimed
```

উদাহরণ:

JAVASCRIPT SOURCE

JAVASCRIPT

```
let user = {
  name: "Ripon"
};
```

Object তৈরি হলো। পরে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
user = null;
```

যদি object-এর আর কোনো reference না থাকে, objectটি garbage collection-এর জন্য eligible হতে পারে।

30.4 Memory Allocation

যখন আমরা data তৈরি করি:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const name = "Ripon";
```

অথবা:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const numbers = [10, 20, 30];
```

অথবা:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const user = {  
  name: "Ripon"  
};
```

JavaScript runtime প্রয়োজন অনুযায়ী memory allocate করে। Conceptually:

ARCHITECTURE & MEMORY GRAPH DIAGRAM

```
name  
↓  
"Ripon"  
  
numbers  
↓  
[10, 20, 30]  
  
user  
↓  
{ name: "Ripon" }
```

30.5 Stack vs Heap

Chapter 29-এ আমরা Call Stack এবং Heap দেখেছিলাম। Memory-এর basic mental model:

STACK VS HEAP CONCEPTUAL ARCHITECTURE

Stack

Execution-related

Primitive values / references
Function frames

Heap

Objects
Arrays
Functions
Dynamic data

তবে একটি গুরুত্বপূর্ণ কথা: > Stack বনাম Heap-এর exact implementation JavaScript specification দ্বারা এভাবে নির্ধারিত নয়। এটি বোঝার জন্য একটি useful conceptual model।

30.6 Primitive Values

JavaScript-এর primitive values:

🔗 ARCHITECTURE & MEMORY GRAPH DIAGRAM

```
string
number
bigint
boolean
undefined
null
symbol
```

উদাহরণ:

JAVASCRIPT SOURCE

JAVASCRIPT

```
let age = 23;
let name = "Ripon";
let isStudent = true;
```

এগুলো primitive values।

30.7 Objects এবং Reference

Object:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const user = {
  name: "Ripon"
};
```

এখানে user variable object-এর সাথে একটি reference-এর মাধ্যমে সম্পর্কিত। Conceptually:

🔗 ARCHITECTURE & MEMORY GRAPH DIAGRAM

user



Object
name: "Ripon"

30.8 Reference কী?

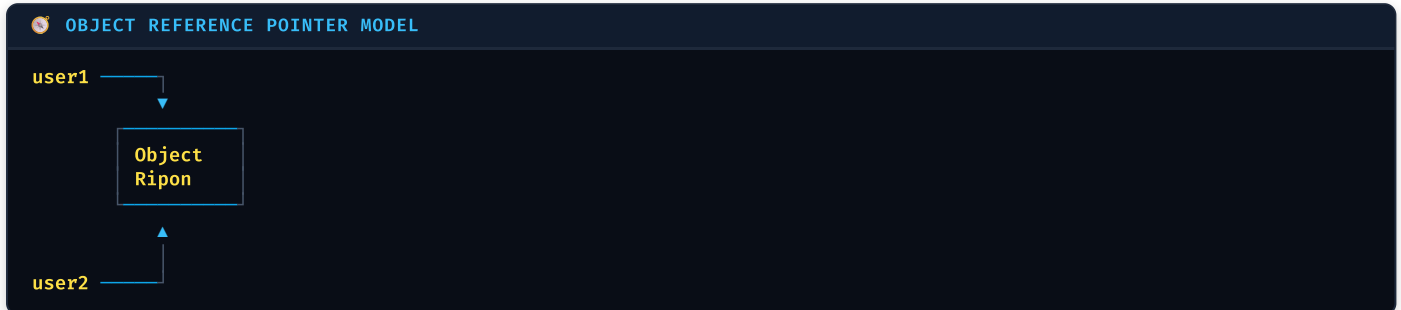
দেখা:

```
JAVASCRIPT SOURCE JAVASCRIPT

const user1 = {
  name: "Ripon"
};

const user2 = user1;
```

এখন:



দুটো variable একই object-কে reference করছে। তাই:

```
JAVASCRIPT SOURCE JAVASCRIPT

user2.name = "Shariar";

console.log(user1.name);
```

► CONSOLE OUTPUT OUTPUT

Shariar

কারণ user1 এবং user2 একই object-এর দিকে reference করছে।

30.9 Primitive Copy vs Reference

Primitive:

```
JAVASCRIPT SOURCE JAVASCRIPT

let a = 10;
let b = a;

b = 20;

console.log(a);
```

► CONSOLE OUTPUT OUTPUT

10

কারণ primitive value copy হয়েছে। অন্যদিকে:

```
JAVASCRIPT SOURCE JAVASCRIPT

const a = {
  value: 10
};

const b = a;

b.value = 20;

console.log(a.value);
```

► CONSOLE OUTPUT OUTPUT

20

কারণ একই object reference করা হয়েছে।

30.10 Reachability — সবচেয়ে গুরুত্বপূর্ণ Concept 🔥

Garbage Collection বোঝার জন্য সবচেয়ে গুরুত্বপূর্ণ concept: > **Reachability** সহজভাবে: > কোনো object যদি program-এর active references থেকে এখনও পৌঁছানো যায়, তাহলে সেটি reachable। আর যদি আর কোনো active reference দিয়ে পৌঁছানো না যায়, তাহলে সেটি garbage collection-এর জন্য eligible হতে পারে।

30.11 Root কী?

Garbage Collector সাধারণত কিছু **roots** থেকে object reachability track করার concept ব্যবহার করে। Examples:

- Global references
- বর্তমানে executing function-এর reachable references
- অন্যান্য runtime roots

Simplified:

🔗 ARCHITECTURE & MEMORY GRAPH DIAGRAM

```
graph TD
  ROOT --> ObjectA[Object A]
  ObjectA --> ObjectB[Object B]
```

যদি Root থেকে B পর্যন্ত পৌঁছানো যায়:

🔗 ARCHITECTURE & MEMORY GRAPH DIAGRAM

B = **Reachable**

30.12 Reachable Example

JAVASCRIPT SOURCE

JAVASCRIPT

```
let user = {
  name: "Ripon"
};
```

Conceptually:

🔗 ARCHITECTURE & MEMORY GRAPH DIAGRAM

```
graph TD
  Global --> user
  user --> Object
```

Object reachable!

30.13 Unreachable Object

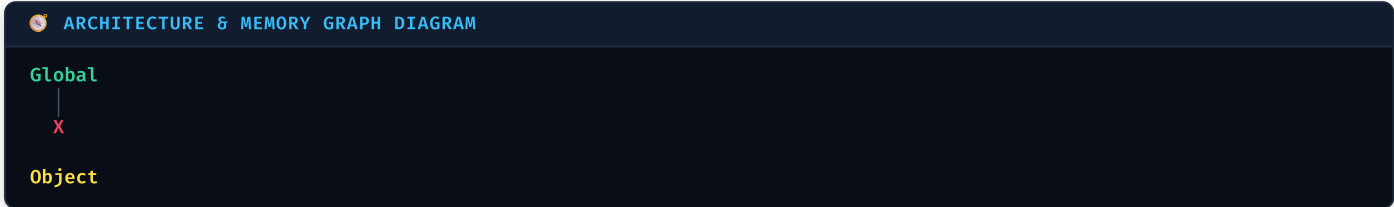
এখন:

```
JAVASCRIPT SOURCE JAVASCRIPT

let user = {
  name: "Ripon"
};

user = null;
```

আগের object-এর দিকে আর user reference নেই। Conceptually:



Object unreachable হয়ে যেতে পারে। তখন Garbage Collector পরে সেটির memory reclaim করতে পারে।

Part 03 — Garbage Collection Engine & Mark-and-Sweep Algorithm (30.14 – 30.21)

30.14 Garbage Collection কী?

Garbage Collector-এর কাজ: > যেসব object আর reachable নয় এবং আর ব্যবহার করা সম্ভব নয়, সেগুলোর memory reclaim করা। সহজভাবে:



30.15 Garbage Collection Manual নয়

তুমি সাধারণত এমন code লিখবে না:

```
JAVASCRIPT SOURCE JAVASCRIPT

free(user);
```

JavaScript-এ সাধারণভাবে এইভাবে manually object free করতে হয় না। বরং:

```
JAVASCRIPT SOURCE JAVASCRIPT

user = null;
```

এর মাধ্যমে reference remove করতে পারো। তারপর object unreachable হলে GC পরে memory reclaim করতে পারে।

30.16 null কি Object Delete করে?

এটা খুব important!

JAVASCRIPT SOURCE

JAVASCRIPT

```
let user = {  
  name: "Ripon"  
};  
  
user = null;
```

null বসানোর সাথে সাথে object অবশ্যই সঙ্গে সঙ্গে delete হয়ে যায়—এমন নয়। বরং:

🕒 ARCHITECTURE & MEMORY GRAPH DIAGRAM

```
user reference removed  
↓  
Object may become unreachable  
↓  
GC may reclaim later
```

Garbage collection-এর timing runtime control করে।

30.17 Garbage Collection সঙ্গে সঙ্গে হয় না

JAVASCRIPT SOURCE

JAVASCRIPT

```
let data = {  
  huge: "some data"  
};  
  
data = null;
```

এখন object unreachable হতে পারে। কিন্তু:

🕒 ARCHITECTURE & MEMORY GRAPH DIAGRAM

```
data = null
```

হওয়ার সাথে সাথেই GC necessarily run করবে—এটা ধরে নেওয়া ঠিক নয়। Garbage Collector কখন memory reclaim করবে সেটা runtime-এর implementation এবং heuristics-এর ওপর নির্ভর করে।

30.18 Mark-and-Sweep

Garbage Collection বোঝার জন্য classic concept: > **Mark-and-Sweep** Simplified process:

🕒 ARCHITECTURE & MEMORY GRAPH DIAGRAM

```
Step 1  
Start from roots  
  
Step 2  
Mark reachable objects  
  
Step 3  
Find unmarked objects  
  
Step 4  
Reclaim their memory
```

30.19 Mark-and-Sweep Example

ধরো:

MARK-AND-SWEEP GC REACHABILITY GRAPH

Root



A



B

C

এখানে:

MARK-AND-SWEEP GC REACHABILITY GRAPH

Root → A → B

তাই:

ARCHITECTURE & MEMORY GRAPH DIAGRAM

A = reachable

B = reachable

C = unreachable

Garbage Collector:

MARK-AND-SWEEP GC REACHABILITY GRAPH

Mark:

A ✓

B ✓

C ✗

Sweep:

C removed/reclaimed

30.20 Circular Reference

আগে পুরনো কিছু garbage collection model-এ circular reference নিয়ে সমস্যা ছিল। কিন্তু modern JavaScript garbage collectors শুধু reference count-এর ওপর নির্ভর করে না; reachability-based tracing techniques ব্যবহার করে। উদাহরণ:

JAVASCRIPT SOURCE

JAVASCRIPT

```
let a = {};  
  
let b = {};  
  
a.other = b;  
b.other = a;
```

এখন:

🕒 ARCHITECTURE & MEMORY GRAPH DIAGRAM

```
a → b  
↑   ↓  
└───┘
```

এটি circular reference। পরে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
a = null;  
b = null;
```

যদি আর কোনো root reference না থাকে:

🕒 MARK-AND-SWEEP GC REACHABILITY GRAPH

```
Root  
X
```

পুরো cycle unreachable হতে পারে এবং garbage collection-এর জন্য eligible হতে পারে।

30.21 Reference Counting কেন যথেষ্ট নয়?

যদি শুধু reference count ব্যবহার করা হতো:

🕒 ARCHITECTURE & MEMORY GRAPH DIAGRAM

```
A → B  
B → A
```

তাহলে:

🕒 ARCHITECTURE & MEMORY GRAPH DIAGRAM

```
A count = 1  
B count = 1
```

দেখে মনে হতে পারে তারা এখনও alive। কিন্তু Root থেকে কেউ তাদের access করতে পারছে না। তাই modern GC reachability-based tracing অনেক বেশি useful।

Part 04 — Memory Leaks, Retained References & Detached DOM Nodes (30.22 – 30.28)

30.22 Memory Leak কী?

Memory Leak হলো এমন situation যেখানে program-এর আর প্রয়োজন নেই এমন memory references-এর কারণে memory unnecessarily retained থাকে। সহজভাবে:

🕒 ARCHITECTURE & MEMORY GRAPH DIAGRAM

```
Memory ব্যবহার হচ্ছে  
↓  
Data আর দরকার নেই  
↓  
কিন্তু reference রয়ে গেছে  
↓  
GC reclaim করতে পারছে না  
↓  
Memory usage বাড়ছে
```

30.23 Memory Leak Example — Global Variable

JAVASCRIPT SOURCE

✗ MEMORY LEAK / ANTI-PATTERN

```
window.myData = [];  
  
function addData(data) {  
  window.myData.push(data);  
}
```

যদি application continuously বড় data যোগ করে:

📊 ARCHITECTURE & MEMORY GRAPH DIAGRAM

```
myData  
↓  
Huge Array  
↓  
More Data  
↓  
More Data
```

এবং application-এর আর প্রয়োজন না থাকলেও global reference ধরে রাখে, memory বাড়তে পারে।

30.24 Event Listener Memory Leak

একটি common frontend problem:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const button = document.querySelector("#button");  
  
function handleClick() {  
  console.log("Clicked");  
}  
  
button.addEventListener("click", handleClick);
```

যদি dynamically created component পরে remove করা হয় কিন্তু listener lifecycle ঠিকভাবে manage না করা হয়, কিছু situation-এ unnecessary references retain হতে পারে। বিশেষ করে SPA/framework applications-এ lifecycle management গুরুত্বপূর্ণ।

30.25 Event Listener Cleanup

যদি listener আর দরকার না হয়:

JAVASCRIPT SOURCE

✓ CLEAN / MEMORY SAFE

```
button.removeEventListener("click", handleClick);
```

Important:

JAVASCRIPT SOURCE

JAVASCRIPT

```
button.addEventListener("click", handleClick);
```

এর পরে remove করতে হলে একই function reference ব্যবহার করা ভালো:

JAVASCRIPT SOURCE

✓ CLEAN / MEMORY SAFE

```
button.removeEventListener("click", handleClick);
```

30.26 Anonymous Function-এর Common Problem

JAVASCRIPT SOURCE

JAVASCRIPT

```
button.addEventListener("click", () => {  
  console.log("Clicked");  
});
```

পরে:

JAVASCRIPT SOURCE

✓ CLEAN / MEMORY SAFE

```
button.removeEventListener("click", () => {  
  console.log("Clicked");  
});
```

এটা listener remove করবে না। কারণ দুইটা function আলাদা।

🕒 ARCHITECTURE & MEMORY GRAPH DIAGRAM

First function ≠ Second function

Better:

JAVASCRIPT SOURCE

✓ CLEAN / MEMORY SAFE

```
function handleClick() {  
  console.log("Clicked");  
}  
  
button.addEventListener("click", handleClick);  
  
button.removeEventListener("click", handleClick);
```

30.27 Detached DOM Node

Frontend-এ একটি গুরুত্বপূর্ণ memory issue: > Detached DOM node ধরো:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const div = document.createElement("div");  
  
const data = {  
  name: "Ripon"  
};  
  
div.data = data;
```

তারপর DOM থেকে node remove করলে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
div.remove();
```

কিন্তু যদি অন্য কোথাও div-এর reference থেকে যায়:

JAVASCRIPT SOURCE

JAVASCRIPT

```
someGlobalReference = div;
```

তাহলে node এবং তার associated references memory-তে retained থাকতে পারে।

30.28 DOM Memory Leak-এর Simple Mental Model

DETACHED DOM NODE RETENTION GRAPH

JavaScript Reference
↓
DOM Node
↓
More Data

DOM থেকে remove করলেই সব memory সঙ্গে সঙ্গে free হবে না। যদি JavaScript থেকে এখনও reference থাকে:

DETACHED DOM NODE RETENTION GRAPH

JS Reference
↓
DOM Node

তাহলে সেটা reachable থাকতে পারে।

Part 05 — Closures & Lexical Scope Memory Lifecycle (30.29 – 30.32)

30.29 Closure এবং Memory

Chapter 26-এ closure দেখেছিলাম। Closure outer variables retain করতে পারে।

JAVASCRIPT SOURCE

JAVASCRIPT

```
function createCounter() {  
  let count = 0;  
  
  return function () {  
    count++;  
    return count;  
  };  
}  
  
const counter = createCounter();
```

এখন counter function-এর মাধ্যমে:

ARCHITECTURE & MEMORY GRAPH DIAGRAM

count

accessible। Conceptually:

CLOSURE LEXICAL SCOPE MEMORY RETENTION

counter
↓
closure
↓
count

তাই closure-এর lifetime বুঝতে memory concepts গুরুত্বপূর্ণ।

30.30 Closure Memory Leak হতে পারে?

Closure নিজে memory leak নয়। কিন্তু closure যদি accidentally বড় object/reference ধরে রাখে এবং সেটা দীর্ঘসময় reachable থাকে, তাহলে unnecessary memory retention হতে পারে। উদাহরণ:

JAVASCRIPT SOURCE

JAVASCRIPT

```
function createHandler() {
  const hugeData = new Array(1000000).fill("data");

  return function () {
    console.log(hugeData.length);
  };
}

const handler = createHandler();
```

handler যতক্ষণ hugeData-কে closure-এর মাধ্যমে retain করে, ততক্ষণ সেই data reachable থাকতে পারে।

30.31 Function Scope এবং Memory

JAVASCRIPT SOURCE

JAVASCRIPT

```
function test() {
  const data = {
    name: "Ripon"
  };

  console.log(data);
}

test();
```

test() execution শেষ হলে সাধারণভাবে data আর reachable না থাকলে তার associated memory GC-এর জন্য eligible হতে পারে।

30.32 But Closure Changes the Lifetime

JAVASCRIPT SOURCE

JAVASCRIPT

```
function test() {
  const data = {
    name: "Ripon"
  };

  return function () {
    return data;
  };
}

const getData = test();
```

এখন:

 CLOSURE LEXICAL SCOPE MEMORY RETENTION

```
getData
↓
closure
↓
data
```

তাই test() শেষ হলেও data retain হতে পারে।

30.33 WeakMap কেন?

Chapter 12-এ আমরা WeakMap দেখেছি। Memory management-এর দিক থেকে WeakMap গুরুত্বপূর্ণ।

JAVASCRIPT SOURCE

✓ CLEAN / MEMORY SAFE

```
const weakMap = new WeakMap();

let user = {
  name: "Ripon"
};

weakMap.set(user, "Private Data");
```

যদি:

JAVASCRIPT SOURCE

JAVASCRIPT

```
user = null;
```

এবং object-এর আর কোনো strong reference না থাকে, WeakMap সেই object-কে এমনভাবে retain করে না যাতে garbage collection বাধ্যতামূলকভাবে বন্ধ হয়ে যায়।

30.34 WeakMap-এর Mental Model

Normal Map:

🔗 ARCHITECTURE & MEMORY GRAPH DIAGRAM

```
Map
↓
Object
↓
Strong reference
```

WeakMap:

🔗 WEAK REFERENCE MEMORY ASSOCIATION

```
WeakMap
↓
Object
↓
Weak association
```

তাই object অন্য কোথাও unreachable হলে WeakMap-এর association garbage collection-এর সাথে disappear হতে পারে।

30.35 WeakSet

একই ধারণা WeakSet-এর ক্ষেত্রেও।

JAVASCRIPT SOURCE

✓ CLEAN / MEMORY SAFE

```
const weakSet = new WeakSet();

let user = {
  name: "Ripon"
};

weakSet.add(user);

user = null;
```

যদি আর কোনো strong reference না থাকে, object garbage collection-এর জন্য eligible হতে পারে।

30.36 WeakMap কোথায় ব্যবহার করা হয়?

একটি practical example:

JAVASCRIPT SOURCE

✓ CLEAN / MEMORY SAFE

```
const metadata = new WeakMap();

function setMetadata(element, data) {
  metadata.set(element, data);
}
```

DOM element:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const button = document.querySelector("button");

setMetadata(button, {
  clicked: true
});
```

যদি button পরে DOM এবং application-এর অন্য strong references থেকে চলে যায়, WeakMap-এর association object lifetime-এর সাথে automatically disappear করতে পারে।

30.37 Cache এবং Memory

Caching performance বাড়ায়:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const cache = new Map();

cache.set("user:1", {
  name: "Ripon"
});
```

কিন্তু unlimited cache:

🔗 ARCHITECTURE & MEMORY GRAPH DIAGRAM

```
Cache
↓
Data
↓
More Data
↓
More Data
↓
Huge Memory
```

হতে পারে। তাই cache design করতে হয়:

- Maximum size
- Expiration
- LRU strategy
- Cleanup
- Appropriate data structure

30.38 Infinite Array — Memory Problem

JAVASCRIPT SOURCE

✗ MEMORY LEAK / ANTI-PATTERN

```
const data = [];

setInterval(() => {
  data.push(new Array(10000).fill("data"));
}, 100);
```

এখানে array ক্রমাগত বড় হচ্ছে।

🕒 ARCHITECTURE & MEMORY GRAPH DIAGRAM

```
data
↓
10000
↓
20000
↓
30000
↓
...
```

Memory usage continuously বাড়তে পারে।

30.39 Memory Leak vs High Memory Usage

দুটো এক নয়।

♦ High Memory Usage

Program legitimately বড় data ব্যবহার করছে।

♦ Memory Leak

Program-এর প্রয়োজন না থাকা data/reference unnecessarily retained হচ্ছে। Example:

🕒 ARCHITECTURE & MEMORY GRAPH DIAGRAM

```
High memory:
Large image processing
```

vs

🕒 ARCHITECTURE & MEMORY GRAPH DIAGRAM

```
Leak:
Old objects never released
```

30.40 Garbage Collection-এর Cost

Garbage Collection automatic হলেও completely free নয়। GC-এর জন্য:

🕒 ARCHITECTURE & MEMORY GRAPH DIAGRAM

```
CPU
+
Memory
+
Time
```

ব্যবহার হতে পারে। যদি application continuously huge temporary objects তৈরি করে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
function process() {
  const huge = new Array(1000000).fill(0);
}
```

অনেক repeated allocation হলে GC pressure বাড়তে পারে।

30.41 Allocation Pressure

ধরো:

JAVASCRIPT SOURCE

JAVASCRIPT

```
for (let i = 0; i < 1000000; i++) {  
  const obj = {  
    value: i  
  };  
}
```

অনেক temporary object তৈরি হচ্ছে। এগুলো পরে unreachable হলে GC reclaim করতে পারে। কিন্তু excessive allocation performance-এর ওপর চাপ তৈরি করতে পারে।

30.42 Object Creation Optimization

প্রয়োজনের বাইরে বারবার object তৈরি করা avoid করা যায়। Bad pattern:

JAVASCRIPT SOURCE

JAVASCRIPT

```
function calculate(user) {  
  return {  
    name: user.name,  
    age: user.age  
  };  
}
```

যদি millions of unnecessary temporary objects তৈরি হয়, performance impact হতে পারে। তবে premature optimization করা উচিত নয়। প্রথমে: > Correctness → Measure → Optimize

30.43 Memory Optimization Rules

♦ Rule 1

Unnecessary global variables avoid করো।

♦ Rule 2

Event listeners cleanup করো।

♦ Rule 3

Large caches unlimited রাখবে না।

♦ Rule 4

Unused references release করো।

♦ Rule 5

Huge objects unnecessarily closure-এ retain করো না।

♦ Rule 6

DOM references lifecycle-এর সাথে manage করো।

♦ Rule 7

Huge arrays continuously grow করতে দিও না।

♦ Rule 8

Performance problem হলে আগে measure করো।

30.44 Chrome DevTools Memory

Browser memory সমস্যা investigate করতে Chrome DevTools ব্যবহার করা যায়। Chrome:

ARCHITECTURE & MEMORY GRAPH DIAGRAM
F12

তারপর:

ARCHITECTURE & MEMORY GRAPH DIAGRAM
Memory

Tabl এখানে সাধারণত:

- Heap Snapshot
- Allocation instrumentation
- Allocation timeline

এর মতো tools পাওয়া যায়।

30.45 Heap Snapshot

Heap Snapshot দিয়ে JavaScript heap-এর object allocation/retention investigate করা যায়। Conceptually:

ARCHITECTURE & MEMORY GRAPH DIAGRAM

```

Before Action
  ↓
Heap Snapshot
  ↓
Perform Action
  ↓
Cleanup
  ↓
Another Snapshot
  ↓
Compare
  
```

যদি object বারবার retained হতে থাকে, leak investigation করা যায়।

30.46 Retainers

Memory debugging-এ একটি গুরুত্বপূর্ণ term: > **Retainer** একটি object কেন memory-তে এখনও আছে? কোন reference সেটাকে ধরে রেখেছে? উদাহরণ:

ARCHITECTURE & MEMORY GRAPH DIAGRAM

```

Global
  ↓
cache
  ↓
user
  ↓
largeObject
  
```

এখানে cache indirectly largeObject-কে retain করছে।

30.47 Garbage Collector কখন Object Collect করবে?

এটা predict করে exact time বলা যায় না। তুমি শুধু বুঝবে:

ARCHITECTURE & MEMORY GRAPH DIAGRAM

```

Object unreachable
  ↓
Eligible for GC
  ↓
GC eventually reclaims
  
```

Eligible ≠ Immediately Collected এই distinction interview-এ গুরুত্বপূর্ণ।

30.48 WeakRef

JavaScript-এ advanced feature:

JAVASCRIPT SOURCE

JAVASCRIPT

WeakRef

এটি object-এর weak reference তৈরি করতে পারে। Example:

JAVASCRIPT SOURCE

✓ CLEAN / MEMORY SAFE

```
let user = {
  name: "Ripon"
};

const weakRef = new WeakRef(user);
```

পরে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const value = weakRef.deref();
```

object এখনও available থাকলে object পাওয়া যেতে পারে। কিন্তু object garbage collected হয়ে গেলে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
weakRef.deref();
```

undefined দিতে পারে।

30.49 WeakRef কেন সাধারণভাবে Avoid করা হয়?

WeakRef advanced use case-এর জন্য। কারণ garbage collection timing deterministic নয়। তাই normal application logic এভাবে design করা উচিত নয়:

🔗 ARCHITECTURE & MEMORY GRAPH DIAGRAM

"GC কখন object remove করবে?"

এর ওপর নির্ভর করে।

30.50 FinalizationRegistry

আরেকটি advanced feature:

JAVASCRIPT SOURCE

JAVASCRIPT

FinalizationRegistry

এটি object garbage-collected হওয়ার পরে cleanup-related notification-এর জন্য ব্যবহার করা যেতে পারে। কিন্তু এটাও: > deterministic cleanup mechanism নয়। Critical resource cleanup-এর জন্য এর ওপর নির্ভর করা উচিত নয়।

30.51 Resource Cleanup বনাম Garbage Collection

খুব গুরুত্বপূর্ণ distinction! Garbage Collection memory reclaim করে। কিন্তু file/network/socket/database resource cleanup আলাদা বিষয় হতে পারে। উদাহরণ:

🔗 ARCHITECTURE & MEMORY GRAPH DIAGRAM

```
Memory cleanup
≠
Business resource cleanup
```

Browser frontend-এ:

🔗 ARCHITECTURE & MEMORY GRAPH DIAGRAM

```
event listener
subscription
timer
WebSocket
observer
```

এগুলোর lifecycle explicitভাবে manage করা লাগতে পারে।

30.52 Timer Cleanup

ধরো:

JAVASCRIPT SOURCE

✗ MEMORY LEAK / ANTI-PATTERN

```
const id = setInterval(() => {
  console.log("Running ... ");
}, 1000);
```

যখন আর দরকার নেই:

JAVASCRIPT SOURCE

✓ CLEAN / MEMORY SAFE

```
clearInterval(id);
```

React-এর মতো framework-এ component unmount হলে এই ধরনের cleanup গুরুত্বপূর্ণ।

30.53 Event Listener Cleanup

JAVASCRIPT SOURCE

✓ CLEAN / MEMORY SAFE

```
function handleClick() {
  console.log("Clicked");
}

button.addEventListener("click", handleClick);

// later
button.removeEventListener("click", handleClick);
```

30.54 Observer Cleanup

যদি তুমি observer ব্যবহার করো:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const observer = new MutationObserver(() => {
  console.log("Changed");
});

observer.observe(element, {
  childList: true
});
```

আর প্রয়োজন না হলে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
observer.disconnect();
```

30.55 WebSocket Cleanup

যেমন:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const socket = new WebSocket("wss://example.com");
```

আর প্রয়োজন না হলে appropriate lifecycle অনুযায়ী:

JAVASCRIPT SOURCE

JAVASCRIPT

```
socket.close();
```

এগুলো memory management-এর পাশাপাশি **resource lifecycle management**।

30.56 React-এর সাথে Memory Management

React application-এ common cleanup:

JAVASCRIPT SOURCE

JAVASCRIPT

```
useEffect(() => {
  const id = setInterval(() => {
    console.log("Running");
  }, 1000);

  return () => {
    clearInterval(id);
  };
}, []);
```

এখানে:

 ARCHITECTURE & MEMORY GRAPH DIAGRAM

```
Component Mount
  ↓
Timer Start
  ↓
Component Unmount
  ↓
Cleanup
  ↓
clearInterval()
```

এটি unnecessary work এবং retained references কমাতে সাহায্য করে।

30.57 Memory Leak Example — React Style

Problematic pattern:

JAVASCRIPT SOURCE

 MEMORY LEAK / ANTI-PATTERN

```
useEffect(() => {
  const id = setInterval(() => {
    fetchData();
  }, 1000);
}, []);
```

যদি effect lifecycle-এর সাথে timer cleanup না করা হয়, component lifecycle অনুযায়ী unnecessary interval running/retention হতে পারে। Better:

JAVASCRIPT SOURCE

JAVASCRIPT

```
useEffect(() => {
  const id = setInterval(() => {
    fetchData();
  }, 1000);

  return () => {
    clearInterval(id);
  };
}, []);
```

30.58 Real-Life Example

ধরো তোমার একটি warehouse আছে।

🕒 ARCHITECTURE & MEMORY GRAPH DIAGRAM

```
Warehouse = Memory
Products = Objects
Warehouse Manager = Garbage Collector
Inventory List = References
```

নতুন product এলো:

🕒 MEMORY LIFECYCLE & RECLAMATION FLOW

Allocate

Product ব্যবহার হচ্ছে:

🕒 ARCHITECTURE & MEMORY GRAPH DIAGRAM

Use

Product আর প্রয়োজন নেই এবং inventory-তেও নেই:

🕒 ARCHITECTURE & MEMORY GRAPH DIAGRAM

Unreachable

Manager:

🕒 ARCHITECTURE & MEMORY GRAPH DIAGRAM

Garbage Collector

পুরনো product সরিয়ে জায়গা free করে। কিন্তু inventory list-এ product-এর reference থাকলে:

🕒 ARCHITECTURE & MEMORY GRAPH DIAGRAM

```
Reference
  ↓
Product
```

manager সেটাকে ফেলে দিতে পারবে না। এটাই memory retention-এর basic idea।

Part 11 — Interview Deep Dives, Golden Rules & Memory Leak Checklist (30.59 – 30.64)

30.59 🔥 Interview Question

♦ প্রশ্ন:

JAVASCRIPT SOURCE

JAVASCRIPT

```
let user = {
  name: "Ripon"
};

let admin = user;

user = null;
```

Object কি garbage collected হবে?

♦ Answer:

না, এই মুহূর্তে **not necessarily** নয়। কারণ:

🕒 ARCHITECTURE & MEMORY GRAPH DIAGRAM

```
admin
  ↓
Object
```

এখনও reference আছে। তাই object reachable।

30.60 আরেকটি

JAVASCRIPT SOURCE

JAVASCRIPT

```
let user = {  
  name: "Ripon"  
};  
  
let admin = user;  
  
user = null;  
admin = null;
```

এখন যদি আর কোনো reference না থাকে:

🕒 MARK-AND-SWEEP GC REACHABILITY GRAPH

```
Object  
  X  
Root
```

Object unreachable হতে পারে এবং garbage collection-এর জন্য eligible হবে।

30.61 Closure Question

JAVASCRIPT SOURCE

JAVASCRIPT

```
function create() {  
  const secret = "Hello";  
  
  return function () {  
    console.log(secret);  
  };  
}  
  
const fn = create();
```

create() execution শেষ হলেও secret কেন থাকতে পারে? কারণ:

🕒 CLOSURE LEXICAL SCOPE MEMORY RETENTION

```
fn  
↓  
Closure  
↓  
secret
```

fn যতক্ষণ reachable এবং closure secret ব্যবহার করতে পারে, secret retained হতে পারে।

30.62 🔥 Memory Management-এর Golden Rule

> Garbage Collector দেখে object reachable কি না; তুমি object আর ব্যবহার না করলেও যদি কোনো active reference সেটাকে ধরে রাখে, তাহলে GC সেটাকে reclaim করতে পারবে না।

30.63 Memory Leak Checklist

যদি application-এর memory ক্রমাগত বাড়তে থাকে, check করো:

🔍 12-POINT MEMORY AUDIT & LEAK CHECKLIST

✓ Global variables

✓ Unlimited cache

✓ setInterval

✓ DOM references

✓ Closures

✓ WebSocket

✓ Large Maps/Sets

✓ Large arrays

✓ Event listeners

✓ setTimeout

✓ Detached DOM nodes

✓ Subscriptions

✓ Observers

✓ Long-lived objects

30.64 🔥 MUST KNOW — মেমরি ম্যানেজমেন্টের প্রধান স্তম্ভসমূহ

চ্যাপ্টার ৩০ থেকে প্রতিটি সফটওয়্যার ইঞ্জিনিয়ারের যেসব বিষয় মুখস্থ নয়, সরাসরি মস্তিষ্কে গেঁথে থাকা বাধ্যতামূলক:

📌 1. Memory Basics

- Memory Allocation (Primitive vs Object)
- Stack (Execution contexts & primitive values)
- Heap (Dynamic collections, objects & arrays)
- References & Memory Addresses
- Reachability & Active Root Graphs

📌 2. Garbage Collection

- Mark-and-Sweep Core Engine Algorithm
- Active Roots Traversal & Mark Bit Setting
- Unreachable vs Nullified References
- Circular References handled cleanly by GC
- GC Runs Asynchronously (Non-deterministic)

📌 3. Memory Leaks Traps

- Accidental Global Variables (window.data)
- Uncleared Timers (setInterval/setTimeout)
- Forgotten Event Listeners on unmounted DOM
- Detached DOM Nodes retained in JS collections
- Closures retaining heavy parent lexical scopes

📌 4. Advanced Diagnostics

- WeakMap & WeakSet (Garbage-collectible keys)
- WeakRef & FinalizationRegistry
- Chrome DevTools Heap Snapshots & Retainers
- Allocation Instrumentation on timeline
- Explicit Resource Cleanup (Freeing buffers)

30. Cheat 🍷 Quick Cheat Sheet — JavaScript Memory & GC Terminology

মেমরি ম্যানেজমেন্টের সর্বাধিক ব্যবহৃত টার্মগুলোর তাৎক্ষণিক অর্থ ও ভূমিকা:

Term	Meaning
Memory Allocation	Data-এর জন্য memory নেওয়া
Reference	Object-এর দিকে reference
Reachable	Root থেকে object-এ পৌঁছানো যায়
Unreachable	আর কোনো active path নেই
Garbage Collector	Unreachable memory reclaim করে
Mark-and-Sweep	Reachable mark, unreachable reclaim
Memory Leak	প্রয়োজনহীন data retained থাকা
Closure	Outer variables retain করতে পারে
Detached DOM	DOM থেকে remove হলেও reference retained
WeakMap	Weakly held object-key association
WeakSet	Weakly held object collection
WeakRef	Weak reference
Heap Snapshot	Memory analysis tool
Retainer	কোন reference object-কে ধরে রেখেছে

মেমরি ম্যানেজমেন্ট ও রিচেবিলিটির খুঁটিনাটি যাচাই করার জন্য নিচে ৬টি প্রোডাকশন ইন্টারভিউ চ্যালেঞ্জের পূর্ণাঙ্গ ব্যাখ্যাসহ সমাধান দেওয়া হলো:

Practice 1: Object Eligibility for Garbage Collection

BEGINNER

? Object কি garbage collection-এর জন্য eligible হতে পারে? কেন?

PRACTICE 1 CODE CHALLENGE

JAVASCRIPT

```
let user = {
  name: "Ripon"
};

user = null;
```

► CONSOLE OUTPUT

OUTPUT

Eligible for Garbage Collection

🔍 Model Solution & Memory Mechanics:

- হ্যাঁ, অবজেক্টটি Garbage Collection-এর জন্য নিশ্চিতভাবে Eligible হবে।
- কারণ: শুরুতে Global/Stack-এর 'user' ভেরিয়েবলটি Memory Heap-এ থাকা '{ name: "Ripon" }' অবজেক্টকে রেফারেন্স করছিল।
- যখন 'user = null' অ্যাসাইন করা হলো, তখন অবজেক্টটির সাথে রুট (Root)-এর একমাত্র রেফারেন্স সংযোগটি বিচ্ছিন্ন হয়ে যায়।
- অবজেক্টটি সম্পূর্ণ 'Unreachable' হয়ে পড়ে। Mark-and-Sweep অ্যালগরিদম পরবর্তী GC সাইকেলে রুট থেকে ট্রান্সার্স করার সময় এই অবজেক্টে পৌঁছাতে পারবে না, ফলে এর দখলকৃত মেমরি রিক্রাইম (মুক্ত) করে দেবে।

Practice 2: Object Reference Copy vs Value Mutability

BEGINNER

? নিচের কোডটি রান করলে আউটপুট কী আসবে এবং মেমরিতে কী ঘটবে?

PRACTICE 2 CODE CHALLENGE

JAVASCRIPT

```
const a = {
  value: 10
};

const b = a;

b.value = 20;

console.log(a.value);
```

► CONSOLE OUTPUT

OUTPUT

20

🔍 Model Solution & Memory Mechanics:

- আউটপুট আসবে '20'।
- মেমরি বিশ্লেষণ: JavaScript-এ অবজেক্ট সরাসরি ভ্যালু হিসেবে কপি হয় না। যখন 'const b = a' লেখা হয়, তখন অবজেক্টের মেমরি অ্যাড্রেস (Reference Pointer) 'b'-তে কপি হয়।
- ফলে 'a' এবং 'b' উভয়েই Memory Heap-এর একই ফিজিক্যাল অবজেক্টকে পয়েন্ট করে থাকে।
- সুতরাং 'b.value = 20' মিউটেট করলে একই অবজেক্টের অভ্যন্তরীণ প্রোপার্টি পরিবর্তিত হয়, তাই 'a.value'-ও ২০ প্রিন্ট করে।

Practice 3: Active Multi-Reference Reachability Check

INTERMEDIATE

? নিচের কোডে user = null করার পরও অবজেক্ট কি reachable থাকবে?

PRACTICE 3 CODE CHALLENGE

JAVASCRIPT

```
let user = {
  name: "Ripon"
};

let admin = user;

user = null;
```

► CONSOLE OUTPUT

OUTPUT

Object remains Reachable (Retained)

🔍 Model Solution & Memory Mechanics:

- হ্যাঁ, অবজেক্টটি এখনো সম্পূর্ণ Reachable থাকবে এবং Garbage Collector এটিকে মুছেবে না।
- কারণ: যদিও 'user = null' করে একটি রেফারেন্স মুছে ফেলা হয়েছে, কিন্তু 'admin' ভেরিয়েবলটি এখনো সেই অবজেক্টের মেমরি অ্যাড্রেস ধরে রেখেছে।
- রুট থেকে অবজেক্টে পৌঁছানোর পথ এখনো অক্ষত আছে: (Global Scope → admin → Heap Object)।
- অবজেক্ট তখনই Unreachable হবে যখন 'admin'-কেও null বা অন্য ভ্যালুতে রি-অ্যাসাইন করা হবে।

Practice 4: Complete Unreachability Point Determination

INTERMEDIATE

? ঠিক কোন লাইনের এক্সিকিউশনের পর অবজেক্টটি GC-এর জন্য eligible হবে?

PRACTICE 4 CODE CHALLENGE

JAVASCRIPT

```
let user = {
  name: "Ripon"
};

let admin = user;

user = null;
admin = null; // ← Critical Transition Point
```

► CONSOLE OUTPUT

OUTPUT

Eligible right after 'admin = null;'

🔍 Model Solution & Memory Mechanics:

- ঠিক 'admin = null;' স্টেটমেন্টটি এক্সিকিউট হওয়ার পরপরই অবজেক্টটি GC-এর জন্য Eligible হবে।
- ধাপ ১: 'user = null' করার পর অবজেক্টের রেফারেন্স কাউন্ট ১ ছিল (admin রেফারেন্স করছিল)।
- ধাপ ২: যখন 'admin = null' কার্যকর হলো, তখন অবজেক্টটির দিকে কোনো সক্রিয় রুট রেফারেন্স অবশিষ্ট থাকল না।
- ফলশ্রুতিতে অবজেক্টটি পুরোপুরি আনরিচিবল গ্রাফ আইল্যান্ডে পরিণত হলো এবং পরবর্তী ইঞ্জিন সুইপ ফেজে মেমরি ফ্রি করার জন্য চূড়ান্তভাবে প্রস্তুত হলো।

Practice 5: Closure Lexical Environment Retention

ADVANCED 🔥

? createCounter() এর এক্সিকিউশন শেষ হওয়ার পরও 'count' ভেরিয়েবল মেমরিতে কীভাবে বাঁচে?

PRACTICE 5 CODE CHALLENGE

JAVASCRIPT

```
function createCounter() {
  let count = 0;

  return function () {
    count++;
    return count;
  };
}

const counter = createCounter();
console.log(counter()); // 1
console.log(counter()); // 2
```

► CONSOLE OUTPUT

OUTPUT

1
2

🔍 Model Solution & Memory Mechanics:

- সাধারণ ফাংশনের ক্ষেত্রে এক্সিকিউশন শেষ হওয়ারামাত্রই কল স্ট্যাক ফ্রেম ধ্বংস হয়ে যায় এবং লোকাল ভেরিয়েবল মুছে যায়।
- কিন্তু এখানে 'createCounter' একটি ইনার ফাংশন রিটার্ন করেছে যা তার প্যারেন্ট স্কোপের 'count' ভেরিয়েবলকে ব্যবহার করে (Closure)।
- JavaScript ইঞ্জিন দেখে যে রিটার্ন করা ফাংশনটি গ্লোবাল ভেরিয়েবল 'counter'-এ অ্যাসাইন করা হয়েছে (Reachable Root)।
- ইঞ্জিন ফাংশনটির Lexical Environment-কে স্ট্যাক থেকে হিপে স্থানান্তরিত করে জিইয়ে রাখে, যতক্ষণ 'counter' ভেরিয়েবলটি মেমরিতে সক্রিয় থাকবে।
- মেমরি সতর্কতা: ক্লোজার যদি ভারী ডেটা রেফারেন্স করে রাখে, তবে কাজ শেষে 'counter = null' না করলে মেমরি রিটেন্ড থাকে।

Practice 6: Timer Interval Memory Leak Audit & Production Lifecycle Cleanup

ADVANCED 🔥

? নিচের কোডে সম্ভাব্য Memory Leak কী এবং কীভাবে ক্লিনআপ করতে হবে?

PRACTICE 6 CODE CHALLENGE

JAVASCRIPT

```
// ❌ Leaky Implementation:
function start() {
  setInterval(() => {
    console.log("Running ... ");
  }, 1000);
}
start();

// ✅ Production Clean Lifecycle Pattern:
let timerId = null;

function startService() {
  if (timerId !== null) return;
  timerId = setInterval(() => {
    console.log("Service pulse active ... ");
  }, 1000);
}

function stopService() {
  if (timerId !== null) {
    clearInterval(timerId);
    timerId = null;
    console.log("Timer destroyed & memory released cleanly.");
  }
}
```

▶ CONSOLE OUTPUT

OUTPUT

```
Service pulse active...
Timer destroyed & memory released cleanly.
```

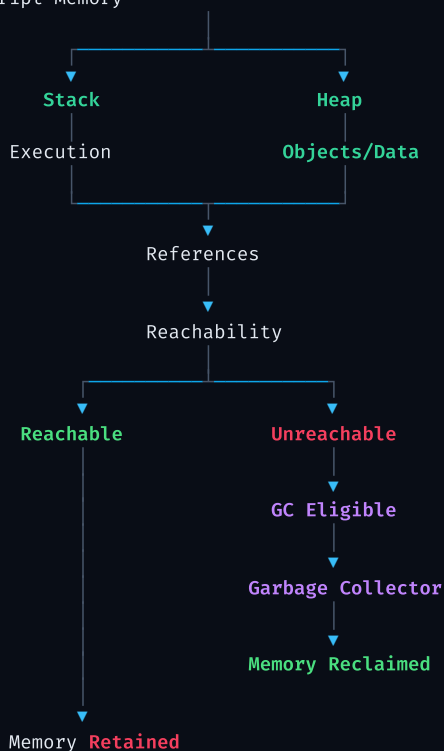
🔍 Model Solution & Memory Mechanics:

১. মেমরি সমস্যা: 'setInterval' ব্রাউজার বা Node.js হোস্ট এনভায়রনমেন্টের ইন্টারনাল টাইমার টেবিলে রেজিস্টার হয়ে থাকে। ফাংশন শেষ হলেও টাইমার অবজেক্ট এবং তার কলব্যাকটি মেমরিতে চিরতরে আটকে থাকে।
২. কলব্যাকের ভেতরে কোনো ভ্যারিয়েবল, অবজেক্ট বা DOM রেফারেন্স থাকলে তারা কখনো GC হবে না।
৩. প্রতিকার: টাইমার আইডিটি সংরক্ষণ করতে হবে এবং কম্পোনেন্ট আনমাউন্ট বা কাজ শেষ হলে 'clearInterval(timerId)' ও 'timerId = null' কল করে হোস্ট মেমরি মুক্ত করতে হবে।
৪. React কম্পোনেন্টে এটি useEffect-এর return cleanup ফাংশনে হ্যান্ডল করা বাধ্যতামূলক।

JavaScript রানটাইমে মেমরি রুপান্তর, রিচেবিলিটি চেক এবং গার্বজ কালেকশন প্রবাহের পূর্ণাঙ্গ মানসিক মানচিত্র:

🕒 JAVASCRIPT MEMORY & GC GLOBAL COORDINATION TREE

JavaScript Memory



★ সবচেয়ে গুরুত্বপূর্ণ ৫টি গোল্ডেন রুল:

- 1 স্বয়ংক্রিয় মেমরি পরিচালনা: JavaScript স্বয়ংক্রিয়ভাবে মেমরি বরাদ্দ ও মুক্ত করে, কোনো ম্যানুয়াল free() প্রয়োজন হয় না।
- 2 রেফারেন্সের মাধ্যমে অবজেক্ট অ্যাক্সেস: অবজেক্ট ভালু আকারে কপি হয় না, কেবল পয়েন্টার রেফারেন্স শেয়ার হয়।
- 3 সক্রিয় অবজেক্ট রিচেল থাকে: রুট (Root) থেকে পৌঁছানো যায় এমন সব ডেটাকে ইঞ্জিন বাঁচিয়ে রাখে।
- 4 আনিরিচেল অবজেক্টই GC যোগ্য: কোনো অবজেক্টের রেফারেন্স চেনা ছিল হলেই কেবল তা গার্বজ কালেকশন উপযোগী হয়।
- 5 মেমরি লিকের মূল কারণ: প্রয়োজনহীন ডেটা যখন ভুলে কোনো সক্রিয় রুট রেফারেন্সের সাথে আবদ্ধ থাকে।

🔥 এক নজরে মূল কথা (Core Essence):

JavaScript-এর Garbage Collector মেমরি নিজে থেকে পরিষ্কার করে, কিন্তু কোনো অপ্রয়োজনীয় অবজেক্ট যদি এখনও কোনো সক্রিয় রেফারেন্সের মাধ্যমে রিচেল থাকে, তাহলে Garbage Collector সেটাকে রিক্লেইম করতে পারে না—এটাই মেমরি ম্যানাজমেন্ট বোঝার মূল চাবিকাঠি।

🚀 Next Chapter Preview:

Chapter 31 — JavaScript Security & Web Security

পরবর্তী চ্যাপ্টারে আমরা শিখব XSS, CSRF, CORS, CSP, DOM-based attacks, Prototype Pollution, Token & Storage Security এবং স্যানিটাইজেশন প্রোডাকশন প্যাটার্ন।